Delivery Route Optimizer

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Koumudi (2520090161)
Rohita (2520090235)

Team Number:

Section: 10

Under the guidance of

Ms K Chandhusha

Assistant Professor, CS&IT

(DEEMED TO BE UNIVERSITY)
BACHUPALLY
Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "**Delivery Route Optimizer"** is a record of Bonafide work of **Koumudi – 2520090161, Rohita – 2520090235,** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artificial Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Koumudi – 2520090161

Rohita – 2520090235

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Delivery Route Optimizer"** is a bonafide work done and submitted by **Koumudi – 2520090161, Rohita – 2520090235** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Ms K Chandhusha, Assistant Professor, CS&IT which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Koumudi – 2520090161

Rohita – 2520090235

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7 - 8
3.	Literature survey	9 - 10
4.	Problem Statement and Objectives	11 - 12
5.	Hardware and Software requirements	13
6.	Implementation	14 - 20
7.	Results & Discussion	21 - 24
8.	Conclusion & Future Scope	25
9.	References	26 - 27

1.ABSTRACT

The **Delivery Route Optimizer** is a web-based application developed to simplify delivery planning by organizing delivery locations, assigning deliveries to vehicles based on their carrying capacities, and generating efficient delivery routes. Manual route planning can be time-consuming and may result in unnecessary travel, increased fuel consumption, and inefficient vehicle utilization. This project demonstrates how data structures and algorithms can be applied to a simplified logistics problem.

The system represents delivery locations as vertices and distances between connected locations as weighted edges in a graph. The Python backend implements the **Nearest Neighbor Greedy Heuristic**, which constructs a route by repeatedly selecting the nearest unvisited delivery location. A capacity-based assignment approach allocates deliveries to vehicles, while the project modules support constraint checking and performance analysis. Estimated travel time and fuel consumption are calculated using configured assumptions.

The frontend is developed using HTML, CSS, and JavaScript, while Python and Flask provide the backend API. Users can manage delivery and vehicle information, request route optimization, and view generated results through a dashboard. The project demonstrates the practical use of graph representation, greedy algorithms, capacity checking, and performance calculations in a delivery-planning application. The generated route is a heuristic solution and is not guaranteed to be globally optimal. Future improvements may include live road distances, traffic information, and advanced optimization algorithms.

Team Name: Delivery Route Optimizer

Team Members:

1. Koumudi, 2520090161
2. Rohita, 2520090235

2. INTRODUCTION

Background

Delivery services are an important part of e-commerce, retail distribution, and logistics operations. As the number of deliveries increases, planning efficient routes becomes more challenging. Delivery vehicles must visit multiple locations while considering travel distance, vehicle capacity, and delivery requirements. Inefficient planning can lead to unnecessary travel, higher fuel consumption, and longer delivery times.

The Delivery Route Optimizer demonstrates how data structures and algorithms can help organize delivery information and generate practical routes. The system provides a web-based interface for entering delivery and vehicle details and displays the results generated by the backend.

Motivation

Planning delivery routes manually becomes difficult when several locations and vehicles are involved. Selecting the order in which locations should be visited and assigning packages to vehicles requires systematic decision-making. A computational approach can reduce repetitive manual work and present route information in an organized format.

The project was developed to apply concepts from Data Structures and Algorithms – III to a practical route-planning problem, particularly graph representation, greedy decision-making, capacity checking, and performance analysis.

Proposed System

The proposed system consists of a frontend dashboard and a Python-based backend. The frontend allows users to manage delivery and vehicle information and request route optimization. The backend represents the delivery network as a weighted graph and uses the Nearest Neighbor Greedy Heuristic to generate routes for assigned delivery locations. Vehicle assignment and performance calculations provide additional information about the generated solution.

When the user requests optimization, the frontend communicates with the Flask API. The backend processes the available data and returns the route results, which are displayed on the dashboard.

Scope

The current project focuses on weighted-graph representation, heuristic route generation, capacity-based vehicle assignment, relevant constraint checking, and performance estimates. The distances are based on the project's configured network and are not live road distances. Travel time and fuel consumption are estimates based on configured assumptions, not real-time traffic or measured vehicle data.

The following sections describe the related work, problem statement, requirements, implementation, results, and possible future enhancements.

3. LITERATURE SURVEY

The Truck Dispatching Problem

Dantzig and Ramser's paper, *The Truck Dispatching Problem* (1959), is an early contribution to vehicle-routing research. It considers how delivery locations can be served by trucks while planning routes and managing distribution requirements. This work provides background for understanding the route-planning and vehicle-assignment problem addressed by the Delivery Route Optimizer.

Vehicle Routing Using Google OR-Tools

Google OR-Tools provides tools for modelling vehicle-routing problems involving multiple vehicles and delivery locations. Its documentation describes how routing models can be constructed and evaluated. This is relevant to our project because it explains the general structure of multi-vehicle route planning, although our current application uses its own Python implementation rather than the OR-Tools solver.

Capacity-Constrained Vehicle Routing

Vehicle-routing problems commonly include a restriction on the total load carried by each vehicle. Capacity constraints help prevent a vehicle from being assigned more packages than it can carry. Our project uses a capacity-based assignment approach to check whether a vehicle has sufficient remaining capacity before accepting a delivery.

Vehicle Routing with Time Windows

Some delivery problems require service within specified time intervals. Time-window routing models include these restrictions when planning feasible routes. This literature is relevant to the future development of our application; time-window support should be claimed as an implemented feature only if it is validated by the current source code.

Nearest Neighbor Heuristic

The Nearest Neighbor method constructs a route by repeatedly selecting the closest unvisited location. Research on the Nearest Neighbor rule for the Traveling Salesperson Problem shows that this method does not always produce the shortest possible tour. Our project uses this heuristic because it provides a simple and practical approach to route generation, while acknowledging that its result is not guaranteed to be globally optimal.

Graph-Based Route Representation

A weighted graph represents locations as vertices and connections between them as edges with associated weights, such as distance. This representation is useful for route-planning problems

because the algorithm can use the edge weights when selecting the next location. The Delivery Route Optimizer uses this concept to store its configured delivery network.

Flask for Backend API Development

Flask is a Python web framework that supports the development of web applications and API endpoints. Its documentation explains how application routes handle incoming HTTP requests. Our project uses Flask to connect the frontend to the backend modules and return route-optimization results.

Python Documentation

The official Python documentation provides information about Python syntax, classes, data structures, and functions. These concepts support the development of the graph, vehicle, optimizer, and performance modules used in our application.

4. Problem Statement and Objectives

Problem Statement: A clear and specific articulation of the problem, outlining its importance.

To develop a delivery route-planning system that assigns deliveries to vehicles based on their carrying capacities and generates efficient routes using data structures and algorithms.

Problem Description: A concise explanation of the issue or challenge that needs to be solved.

Manual delivery planning can lead to inefficient routes, increased travel distance, and unnecessary fuel consumption. A systematic approach is required to assign deliveries to vehicles and generate suitable routes while considering vehicle capacity constraints.

Objectives:

Identify the Project's Purpose:

The purpose of the Delivery Route Optimizer is to simplify delivery planning by applying graph-based representation and greedy route optimization techniques. The system aims to generate practical delivery routes, manage vehicle capacities, and estimate route performance through a web-based dashboard.

The main objectives of the project are:

1. To represent delivery locations and distances using a weighted graph.
2. To implement the Nearest Neighbor Greedy Heuristic for route generation.
3. To assign deliveries to vehicles based on their carrying capacities.
4. To check relevant vehicle capacity and route constraints.
5. To calculate estimated travel distance, travel time, and fuel consumption.
6. To develop a dashboard for managing delivery and vehicle information.

7. To connect the frontend with the Python Flask backend.
8. To display generated routes and performance results in an organized format.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- **Processor:** A modern Intel or AMD processor capable of running the development environment.
- **RAM:** 4 GB minimum; 8 GB recommended.
- **Storage:** Sufficient space for the project files, Python environment, and development tools.
- **Display:** A laptop or desktop display for viewing the dashboard and code.
- **Input Devices:** Keyboard and mouse or touchpad.
- **Network:** Localhost access for running the frontend and backend on the same computer.

Software Requirements:

- **Operating System:** Windows or another operating system that supports Python.
- **Frontend Technologies:** HTML5, CSS3, and JavaScript.
- **Programming Language:** Python.
- **Backend Framework:** Flask.
- **Cross-Origin Requests:** Flask-CORS.
- **Code Editor:** Visual Studio Code or another suitable code editor.
- **Web Browser:** Google Chrome or another modern browser.
- **Python Environment:** Python virtual environment (venv).
- **Communication:** HTTP requests between the frontend and Flask API.
- **Version Control:** Git, if used to manage the project source code.

6. IMPLEMENTATION

The Delivery Route Optimizer is developed using HTML, CSS, and JavaScript for the frontend and Python with Flask for the backend. The backend processes delivery and vehicle information, represents the delivery network as a weighted graph, generates routes using the Nearest Neighbour Greedy Heuristic, and provides performance calculations. The frontend communicates with the backend through API requests and displays the results on the dashboard.

Graph Representation (graph.py)

The graph module represents the delivery network using a weighted graph. Each vertex represents a delivery location, and each weighted edge represents the distance between two connected locations. This representation stores the distance information required by the route optimization algorithm.

app.py graph.py

backend > graph.py

```python
class DeliveryGraph:
    """Weighted adjacency-list graph for the delivery network."""

    def __init__(self):
        self.graph = {}

    def add_location(self, location):
        self.graph.setdefault(location, {})

    def add_route(self, a, b, distance_km):
        self.add_location(a)
        self.add_location(b)
        self.graph[a][b] = float(distance_km)
        self.graph[b][a] = float(distance_km)


delivery_graph = DeliveryGraph()

# Demonstration road network using recognizable Hyderabad delivery areas.
# Distances are project-network values, not live map distances.
delivery_graph.add_route("Warehouse", "Hafeezpet", 8)
delivery_graph.add_route("Warehouse", "Miyapur", 10)
delivery_graph.add_route("Warehouse", "Kondapur", 15)
delivery_graph.add_route("Warehouse", "Gachibowli", 22)
delivery_graph.add_route("Warehouse", "Madhapur", 20)

delivery_graph.add_route("Hafeezpet", "Miyapur", 5)
delivery_graph.add_route("Hafeezpet", "Kondapur", 9)
delivery_graph.add_route("Hafeezpet", "Madhapur", 14)

delivery_graph.add_route("Miyapur", "Kondapur", 8)
delivery_graph.add_route("Miyapur", "Gachibowli", 17)

delivery_graph.add_route("Kondapur", "Gachibowli", 9)
delivery_graph.add_route("Kondapur", "Madhapur", 7)
delivery_graph.add_route("Gachibowli", "Madhapur", 6)

```

Figure 1: Graph representation of delivery locations and distances.

Route Optimization (optimizer.py)

The route optimization module implements the Nearest Neighbor Greedy Heuristic. Starting from the warehouse, it repeatedly selects the nearest unvisited location among the assigned delivery stops. The process continues until the required stops have been visited. This heuristic provides a practical route-generation approach but does not guarantee the globally shortest route.

app.py optimizer.py

backend > optimizer.py

```python
class RouteOptimizer:
    """Nearest Neighbor heuristic on a weighted adjacency-list graph."""

    def __init__(self, graph, average_speed_kmh=40):
        self.graph = graph
        self.average_speed_kmh = float(average_speed_kmh)

    @staticmethod
    def to_minutes(value, default=0):
        try:
            h, m = str(value).split(":")[:2]
            return int(h) * 60 + int(m)
        except (ValueError, TypeError):
            return default

    @staticmethod
    def to_hhmm(minutes):
        minutes = int(round(minutes)) % (24 * 60)
        return f"{minutes // 60:02d}:{minutes % 60:02d}"

    def nearest_neighbor(self, start, deliveries, dispatch_minutes=9 * 60):
        if start not in self.graph:
            return [start, start], 0, 0, [], [], None

        grouped = {}
        for delivery in deliveries:
            grouped.setdefault(delivery["location"], []).append(delivery)

        unvisited = set(grouped)
        current = start
        route = [start]
        total_distance = 0.0
        current_minutes = float(dispatch_minutes)
        timeline = []
        violations = []

        while unvisited:
            candidates = []

            for location in unvisited:
                edge_distance = self.graph.get(current, {}).get(location)
```

app.py optimizer.py

backend > optimizer.py

```python
class RouteOptimizer:
    def nearest_neighbor(self, start, deliveries, dispatch_minutes=9 * 60):
        while unvisited:

            for location in unvisited:
                edge_distance = self.graph.get(current, {}).get(location)
                if edge_distance is None:
                    continue

                travel_minutes = edge_distance / self.average_speed_kmh * 60
                arrival = current_minutes + travel_minutes
                items = grouped[location]
                starts = [self.to_minutes(d.get("time_start", "09:00"), 9 * 60) for d in items]
                ends = [self.to_minutes(d.get("time_end", "18:00"), 18 * 60) for d in items]
                window_start = max(starts)
                window_end = min(ends)
                overlap_ok = window_start <= window_end
                service_start = max(arrival, window_start)
                feasible = overlap_ok and service_start <= window_end

                candidates.append((
                    0 if feasible else 1,
                    edge_distance,
                    window_end,
                    location,
                    arrival,
                    window_start,
                    overlap_ok,
                ))

            if not candidates:
                break

            # Prefer a time-window-feasible stop, then the nearest stop.
            candidates.sort(key=lambda item: (item[0], item[1], item[2], item[3]))
            _, distance, window_end, location, arrival, window_start, overlap_ok = candidates[0]

            current_minutes = max(arrival, window_start)
            on_time = overlap_ok and current_minutes <= window_end
            if not on_time:
                violations.append(location)
```

```
            on_time = overlap_ok and current_minutes <= window_end
            if not on_time:
                violations.append(location)

            total_distance += distance
            route.append(location)
            timeline.append({
                "location": location,
                "arrival": self.to_hhmm(current_minutes),
                "window": f"{self.to_hhmm(window_start)}–{self.to_hhmm(window_end)}",
                "on_time": on_time,
            })

            current = location
            unvisited.remove(location)

        if current in self.graph and start in self.graph[current]:
            return_distance = self.graph[current][start]
            total_distance += return_distance
            current_minutes += return_distance / self.average_speed_kmh * 60
            route.append(start)

        elapsed_minutes = max(0, current_minutes - dispatch_minutes)
        completion_clock = self.to_hhmm(current_minutes)
        return (
            route,
            round(total_distance, 2),
            round(elapsed_minutes),
            timeline,
            sorted(set(violations)),
            completion_clock,
        )
```

Figure 2: Implementation of the Nearest Neighbour route-generation algorithm.

Vehicle Assignment (vehicle.py)

The vehicle module stores vehicle information, including vehicle identifiers, carrying capacities, and assigned deliveries. Before a package is assigned, the system checks whether the vehicle has sufficient remaining capacity. The assignment process uses a sequential, first-fit-style approach to select a suitable vehicle.

app.py vehicle.py

backend > vehicle.py

```python
class Vehicle:
    def __init__(self, vehicle_id, capacity):
        self.vehicle_id = vehicle_id
        self.capacity = float(capacity)
        self.deliveries = []
        self.total_load = 0.0

    def can_accept(self, weight):
        return self.total_load + float(weight) <= self.capacity

    def add_delivery(self, delivery):
        weight = float(delivery["weight"])
        if self.can_accept(weight):
            self.deliveries.append(delivery)
            self.total_load += weight
            return True
        return False


class VehicleAssignment:
    """Capacity-constrained first-fit assignment, with heavier packages first."""

    def __init__(self, vehicles):
        self.vehicles = vehicles

    def assign(self, deliveries):
        unassigned = []
        for delivery in sorted(deliveries, key=lambda d: float(d["weight"]), reverse=True):
            placed = False
            for vehicle in self.vehicles:
                if vehicle.add_delivery(delivery):
                    placed = True
                    break
            if not placed:
                unassigned.append(delivery)
        return unassigned
```

Figure 3: Vehicle capacity checking and delivery assignment.

Backend Integration (app.py)

The Flask backend connects the frontend with the project's Python modules through API endpoints. It receives requests from the frontend, processes the supplied information, and returns the generated route results and other supported dashboard data.

app.py vehicle.py

backend > app.py

```python
from flask import Flask, jsonify, request
from flask_cors import CORS

from graph import delivery_graph
from optimizer import RouteOptimizer
from vehicle import Vehicle, VehicleAssignment
from constraints import ConstraintChecker
from performance import PerformanceAnalyzer

app = Flask(__name__)
CORS(app)

DEFAULT_DELIVERIES = [
    {"location": "Hafeezpet", "weight": 20, "time_start": "09:00", "time_end": "11:00"},
    {"location": "Miyapur", "weight": 30, "time_start": "10:00", "time_end": "12:00"},
    {"location": "Kondapur", "weight": 25, "time_start": "11:00", "time_end": "13:00"},
    {"location": "Gachibowli", "weight": 35, "time_start": "12:00", "time_end": "14:00"},
]

DEFAULT_VEHICLES = [
    {"vehicle_id": "V-01", "capacity": 100},
    {"vehicle_id": "V-02", "capacity": 80},
]


def normalize_deliveries(raw):
    result = []
    valid_locations = set(delivery_graph.graph) - {"Warehouse"}
    for index, item in enumerate(raw or []):
        location = str(item.get("location", "")).strip()
        if location not in valid_locations:
            continue
        try:
            weight = float(item.get("weight", 0))
        except (TypeError, ValueError):
            continue
        if weight <= 0:
            continue
        result.append({
            "location": location,
            "weight": weight,
```

app.py vehicle.py

backend > app.py

```python
def normalize_vehicles(raw):
    for item in raw or []:
        if vehicle_id and capacity > 0:
    return result


def build_result(deliveries=None, vehicles_data=None):
    deliveries = normalize_deliveries(
        DEFAULT_DELIVERIES if deliveries is None else deliveries
    )
    vehicles_data = normalize_vehicles(
        DEFAULT_VEHICLES if vehicles_data is None else vehicles_data
    )

    vehicles = [Vehicle(v["vehicle_id"], v["capacity"]) for v in vehicles_data]
    unassigned = VehicleAssignment(vehicles).assign(deliveries)
    optimizer = RouteOptimizer(delivery_graph.graph, average_speed_kmh=40)
    checker = ConstraintChecker()

    vehicle_results = []
    total_distance = 0.0
    total_fuel = 0.0
    completion_times = []
    feasible_count = 0

    for vehicle in vehicles:
        route, distance, elapsed, timeline, violations, completion_clock = optimizer.nearest_neighbor(
            "Warehouse", vehicle.deliveries
        )
        checks = checker.validate(vehicle, route, violations)
        if checks["feasible"]:
            feasible_count += 1

        fuel = round(distance / 15, 2)
        total_distance += distance
        total_fuel += fuel
        completion_times.append(elapsed)

        vehicle_results.append({
            "vehicle_id": vehicle.vehicle_id,
```

Problems Output Debug Console Terminal Ports

Figure 4: Flask backend and API integration.

The performance and constraint modules support additional calculations and validation. Their output depends on the actual logic implemented in the current project files.

7. RESULTS & DISCUSSION

The Delivery Route Optimizer provides a web-based interface for managing delivery information, viewing available vehicles, generating routes, and examining estimated route performance. The frontend communicates with the Python Flask backend to process the supplied data and display the results.

Dashboard

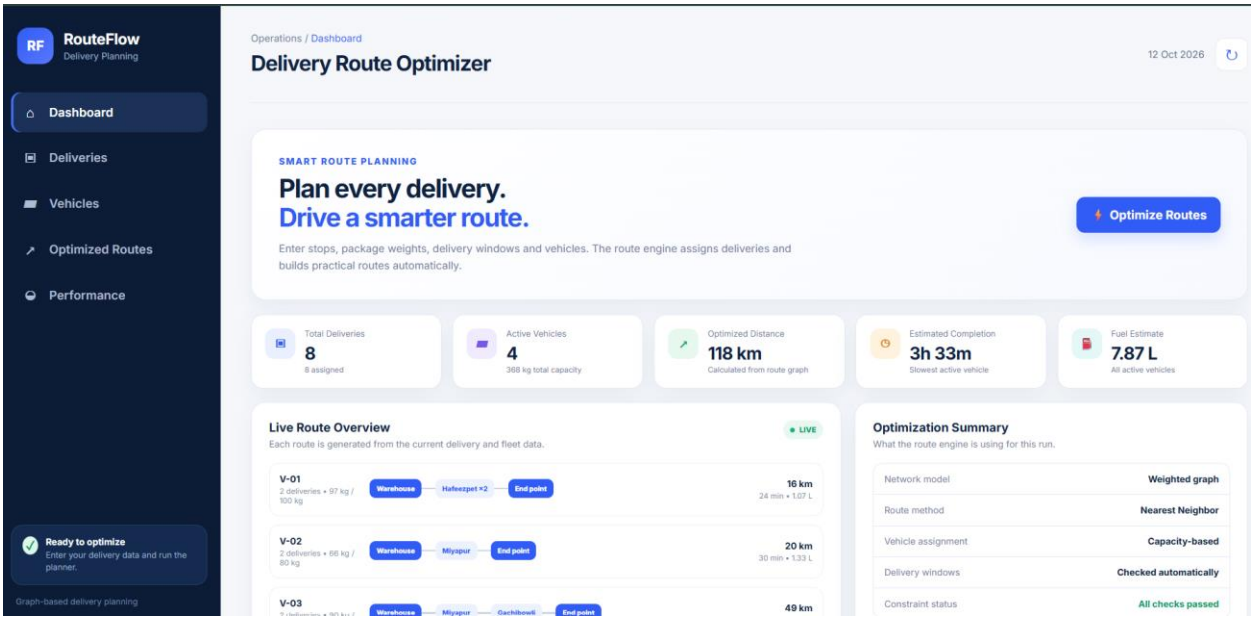


Figure 5: Main dashboard of the Delivery Route Optimizer.

The dashboard presents an overview of the delivery-planning information and route-performance estimates. It provides access to the delivery, vehicle, optimized-route, and performance sections.

Delivery Management

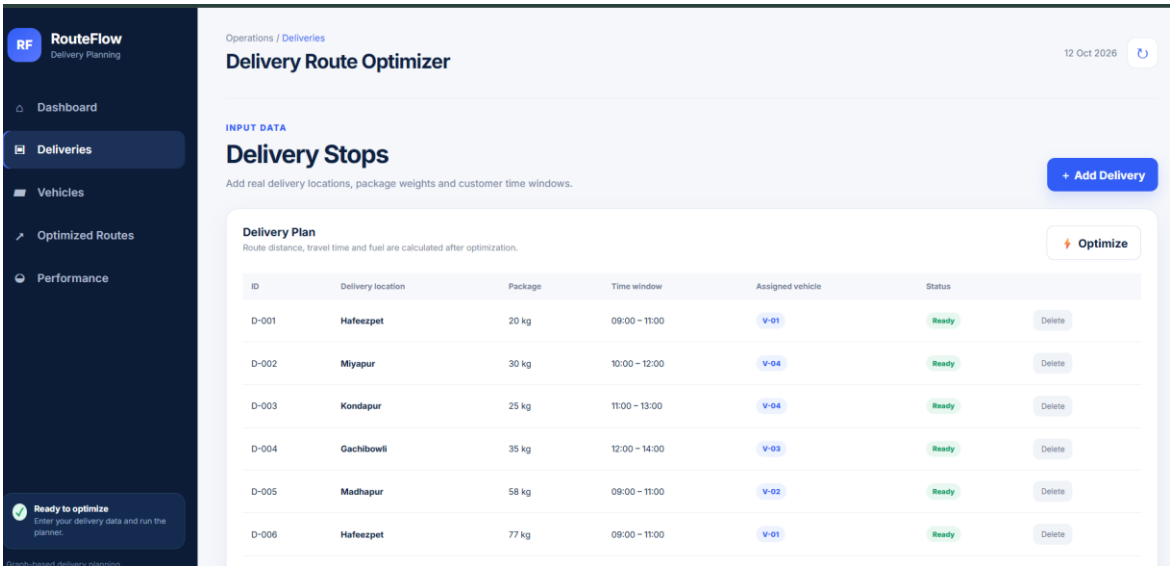


Figure 6: Delivery information displayed in the application.

The delivery section allows users to view and manage delivery information, such as customer details, locations, and package weights. The available delivery data is used during vehicle assignment and route generation.

Vehicle Management

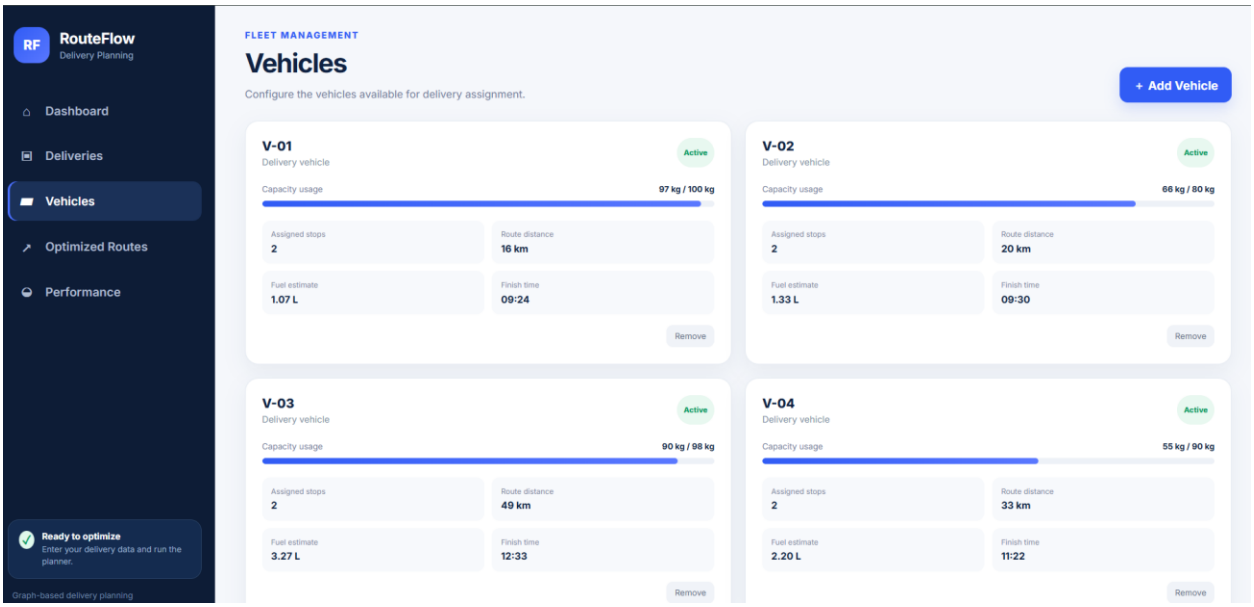


Figure 7: Vehicle information and carrying capacities.

The vehicle section displays the available vehicles and their configured carrying capacities. This information supports the capacity-based assignment of deliveries.

Optimized Routes

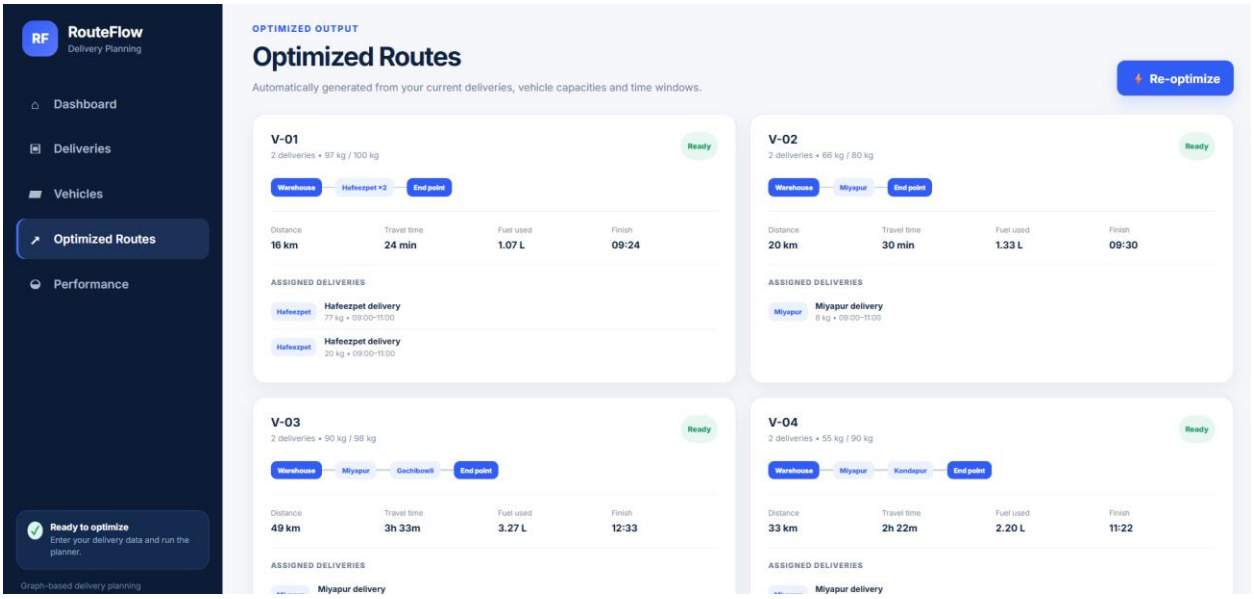


Figure 8: Routes generated by the Nearest Neighbour Greedy Heuristic.

The optimized-routes section displays the routes generated for assigned delivery locations. The results depend on the configured graph distances and the data supplied to the backend.

Performance Analysis

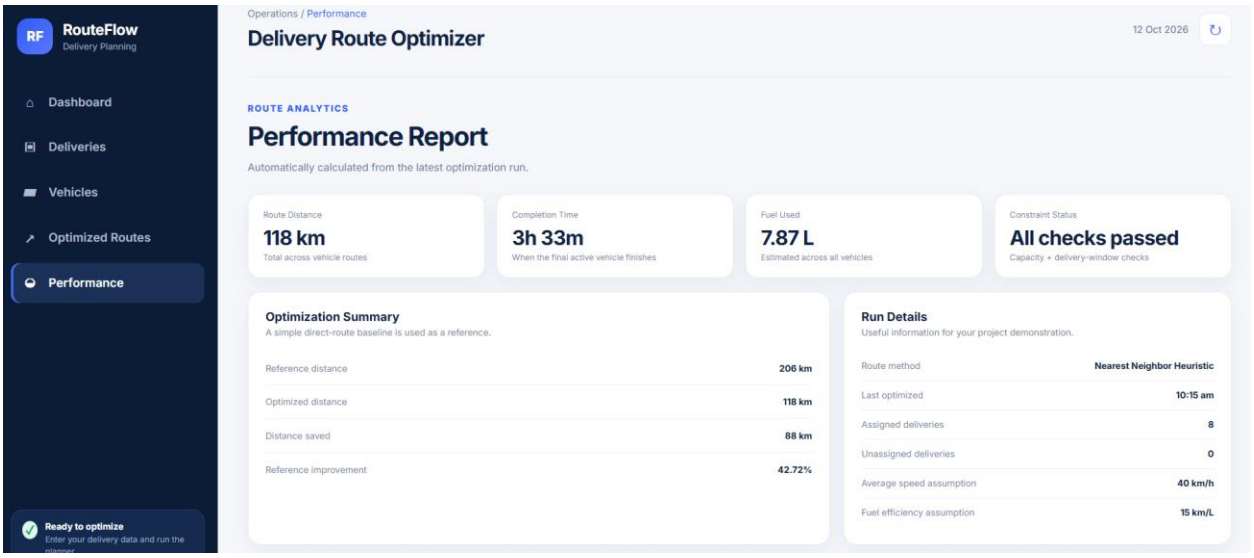


Figure 9: Estimated route-performance metrics.

The performance section presents the distance and estimated travel-time and fuel-consumption metrics calculated by the application. These estimates depend on the configured assumptions and are not measurements from live traffic or vehicle sensors.

Discussion

The application demonstrates how a weighted graph and a greedy heuristic can be integrated into a web-based delivery-planning system. Capacity-based assignment and performance calculations provide additional information about the generated routes. The Nearest Neighbour heuristic does not guarantee the globally shortest route, and the accuracy of the displayed estimates depends on the configured network and calculation assumptions.

8. CONCLUSION & FUTURE SCOPE

The Delivery Route Optimizer demonstrates the application of data structures and algorithms to a practical delivery-planning problem. The system represents delivery locations and their distances using a weighted graph and uses the Nearest Neighbor Greedy Heuristic to generate delivery routes. The vehicle assignment module considers carrying capacities, while the performance calculations provide estimates of travel distance, travel time, and fuel consumption.

The project combines an HTML, CSS, and JavaScript frontend with a Python Flask backend. It allows users to manage delivery and vehicle information and view the generated results through a dashboard. The project demonstrates how graph representation, greedy decision-making, capacity checking, and modular software design can be applied to delivery planning.

The generated route is a heuristic solution and is not guaranteed to be globally optimal. The current system uses configured distances and fixed assumptions for its performance estimates. Nevertheless, it provides a practical academic demonstration of DSA concepts in a delivery-planning application.

Future Scope

The project can be extended in the following ways:

- Live Map Integration: Integrate a mapping service to obtain actual road distances and travel times.
- Traffic-Aware Routing: Include traffic conditions and road restrictions when generating routes.
- Advanced Optimization: Compare the Nearest Neighbor heuristic with other optimization methods, such as local search, simulated annealing, or genetic algorithms.
- Improved Vehicle Assignment: Develop methods that balance vehicle loads and route distances more effectively.
- Time-Window Scheduling: Incorporate customer delivery intervals and service times into route planning.
- Vehicle-Specific Fuel Estimates: Use individual vehicle fuel-efficiency data to improve consumption estimates.
- Route Visualization: Display delivery locations and routes on an interactive map.
- Performance Evaluation: Compare multiple route-generation methods using consistent datasets and performance metrics.
- Larger Dataset Testing: Evaluate the system with more vehicles and delivery locations.

- Deployment: Host the application on a server so that it can be accessed outside the local development environment.

9.REFERENCES

1. G. B. Dantzig and J. H. Ramser, "The Truck Dispatching Problem," *Management Science*, vol. 6, no. 1, pp. 80–91, 1959. Available: https://doi.org/10.1287/mnsc.6.1.80
2. Google for Developers, "Vehicle Routing," *Google OR-Tools Documentation*. Available: https://developers.google.com/optimization/routing
3. Google for Developers, "Common Routing Tasks," *Google OR-Tools Documentation*. Available: https://developers.google.com/optimization/routing/routing_tasks
4. C. A. J. Hurkens and G. J. Woeginger, "On the Nearest Neighbor Rule for the Traveling Salesman Problem," *Operations Research Letters*, vol. 32, no. 1, pp. 1–4, 2004. Available: https://doi.org/10.1016/S0167-6377(03)00093-2

5. Google OR-Tools, “Vehicle Routing Solver Documentation,” GitHub repository. Available: https://github.com/google/or-tools
6. Pallets Projects, “Flask Documentation.” Available: https://flask.palletsprojects.com/en/stable/
7. Pallets Projects, “Quickstart,” *Flask Documentation*. Available: https://flask.palletsprojects.com/en/stable/quickstart/
8. Python Software Foundation, “Python Documentation.” Available: https://docs.python.org/3/